UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Hospital Management System

Project Report

Submitted To-

Mr. Shalabh Bhatiya

Submitted by

Name - Swayam
UID - 25MCA20015
Section - 25MCA_1-A

in partial fulfilment for the award of the degree of

Master of Computer Application

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Chandigarh University

Declaration

I hereby declare that the project report entitled **"Hospital Management System"** submitted in partial fulfilment of the requirements for the degree of Master of Computer Application is my original work and has not been submitted previously to any other institution or university for any degree or diploma.

I have carried out the work presented in this project under the guidance of **Mr. Shalabh Bhatiya**. All the work presented here is genuine to the best of my knowledge, and any references used have been appropriately acknowledged.

I am fully responsible for the authenticity of the content and any inaccuracies in the report.

Acknowledgement

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project on the **Hospital Management System**.

I am especially thankful to my instructor **Mr. Shalabh Bhatiya** for his continuous guidance, valuable suggestions, and support throughout the development of this project. His insights helped me understand database management concepts and their real-world applications.

I would also like to thank my friends and family for their encouragement and support during this project.

This project has been a great learning experience, helping me bridge the gap between theoretical knowledge and practical implementation.

.

Roshan Kumar Singh

Table of Content

1. **Introduction**
2. **Objective**
3. **Software and Hardware requirements**
4. **Implementation**
5. **Key Functionalities**
6. **Results and Screenshots**
7. **Future Scope**
8. **Conclusion**
9. **References**

Introduction

In modern healthcare systems, efficient management of patient data, doctor schedules, appointments, and medical records is essential for providing quality services. Traditional manual systems are inefficient, error-prone, and difficult to maintain.

The Hospital Management System is designed as a comprehensive database solution using PostgreSQL to manage hospital operations efficiently. The system organizes data into structured relational tables and automates various processes such as appointment booking, medical record creation, and prescription management.

This project demonstrates the practical implementation of advanced Database Management System (DBMS) concepts such as schema design, normalization, constraints, joins, views, functions, stored procedures, triggers, and advanced SQL queries.

The system is designed with scalability and real-world applicability in mind, making it suitable for managing large healthcare datasets efficiently.

Objectives

The objectives of this project are:

- To design a structured relational database using PostgreSQL
- To implement normalization techniques up to 3NF
- To establish relationships using primary and foreign keys
- To ensure data integrity using constraints and validations
- To implement stored procedures for business logic
- To use triggers for automation and auditing
- To create views for simplified data retrieval
- To implement functions for reusable logic
- To perform advanced SQL queries for analysis
- To demonstrate real-world DBMS applications

Software Requirements

The system is developed using the following software tools and technologies:

- **Operating System:**
  Windows 10 or above
- **Language:**
  SQL (PL/pgSQL)
- **IDE:**
  pgAdmin

Hardware Requirements

The system requires basic hardware configuration to run efficiently:

- **Processor:**
  Intel Core i3 or higher
- **RAM:**
  Minimum 4 GB

Implementation

The system is implemented using a modular database design approach.

Schema Design

A dedicated schema (hms) is created to organize all database objects. This improves maintainability and separation of concerns.

Tables and Relationships

The system includes the following tables:

- **Departments** – Stores department details
- **Doctors** – Stores doctor information
- **Patients** – Stores patient details
- **Appointments** – Manages patient-doctor interactions
- **Medical Records** – Stores diagnosis and treatment history
- **Prescriptions** – Stores medicines prescribed
- **Doctor Schedule** – Manages availability
- **Appointment Audit** – Tracks status changes

Relationships are established using **foreign keys** to ensure referential integrity.

Normalization

The database is normalized up to **Third Normal Form (3NF)**:

- Eliminates redundancy
- Ensures efficient storage
- Maintains data consistency

Views

Two major views are created:

- **vw_appointment_details** : Combines patient, doctor, and department data
- **vw_patient_summary** : Provides patient statistics

Views simplify complex queries and improve readability.

Functions

Several reusable functions are implemented:

- **fn_patient_age():** Calculates patient age
- **fn_count_patient_appointments():** Counts appointments
- **fn_next_available_slot():** Finds next available doctor slot
- **fn_patient_prescriptions():** Returns prescription history

These functions improve modularity and reusability.

Stored Procedures

Stored procedures handle core business logic:

- **sp_book_appointment**: Validates and books appointment
- **sp_cancel_appointment**: Cancels appointment
- **sp_complete_appointment**: Marks appointment complete and creates record
- **sp_add_prescription**: Adds prescription
- **sp_deactivate_doctor**: Deactivates doctor and cancels future appointments

These procedures ensure controlled and validated data operations.

Triggers

Triggers are used for automation and enforcement:

- **Audit Trigger**: Logs appointment status changes
- **Weekend Restriction Trigger**: Prevents weekend bookings
- **Auto No-Show Trigger**: Updates status automatically
- **Medical Record Lock Trigger**: Prevents modification
- **Prescription Timestamp Trigger**: Auto sets timestamp

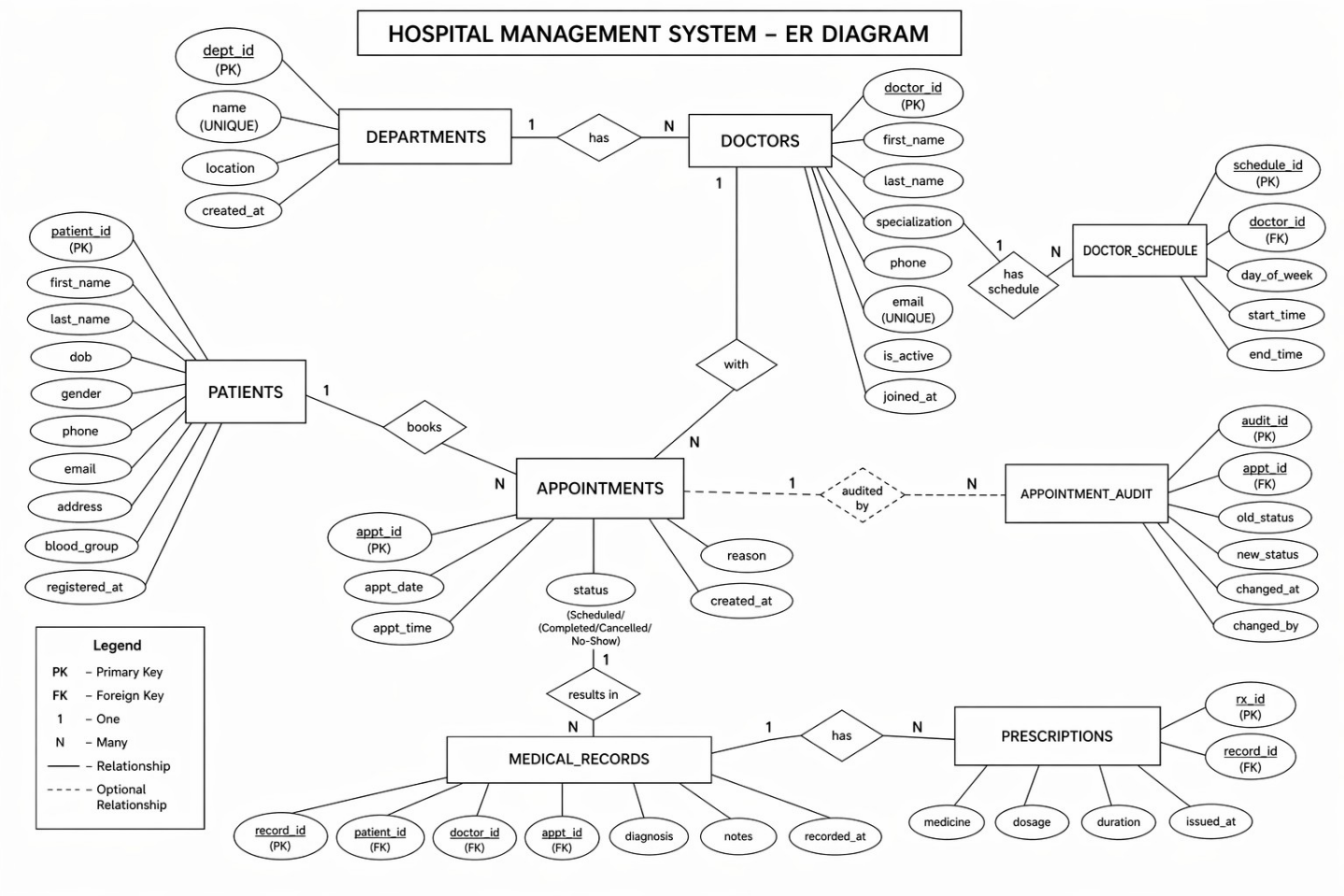
Advanced Queries

The project includes advanced SQL queries such as:

- JOIN queries for relational data
- Aggregate functions for analysis
- Window functions (DENSE_RANK, running totals)
- CTE (Common Table Expressions)
- Filtering and grouping queries

These queries demonstrate strong SQL proficiency.

ER Diagram


HOSPITAL MANAGEMENT SYSTEM – ER DIAGRAM
DEPARTMENTS
dept_id (PK)
name (UNIQUE)
location
created_at
has
1
N
DOCTORS
doctor_id (PK)
first_name
last_name
specialization
phone
email (UNIQUE)
is_active
joined_at
has schedule
DOCTOR_SCHEDULE
schedule_id (PK)
doctor_id (FK)
day_of_week
start_time
end_time
PATIENTS
patient_id (PK)
first_name
last_name
dob
gender
phone
email
address
blood_group
registered_at
books
with
APPOINTMENTS
appt_id (PK)
appt_date
appt_time
status
(Scheduled/ (Completed/Cancelled/ No-Show)
reason
created_at
audited by
APPOINTMENT_AUDIT
audit_id (PK)
appt_id (FK)
old_status
new_status
changed_at
changed_by
results in
MEDICAL_RECORDS
record_id (PK)
patient_id (FK)
doctor_id (FK)
appt_id (FK)
diagnosis
notes
recorded_at
has
PRESCRIPTIONS
rx_id (PK)
record_id (FK)
medicine
dosage
duration
issued_at
Legend
PK – Primary Key
FK – Foreign Key
1 – One
N – Many
—— – Relationship
- - - - – Optional Relationship

Key Functionalities

The Hospital Management System provides several important functionalities based on relational database concepts:

- **Structured Data Storage:**
  Data is organized into multiple related tables such as patients, doctors, appointments, medical records, and prescriptions, ensuring efficient storage and retrieval.
- **Appointment Management:**
  Appointments are created and managed using stored procedures with validation for doctor availability and time conflicts.
- **Doctor Scheduling:**
  The system maintains doctor availability using a schedule table, allowing better planning and management of appointments.
- **Medical Record Management:**
  Each completed appointment generates a medical record containing diagnosis and treatment details.
- **Prescription Handling:**
  Prescriptions are stored with details such as medicine, dosage, and duration, linked to patient records.
- **Data Integrity and Constraints:**
  The system uses primary keys, foreign keys, and check constraints to maintain accurate and consistent data.
- **Automation using Triggers:**
  Triggers are used for tasks like audit logging, preventing invalid bookings, and updating appointment status automatically.
- **Data Retrieval and Reporting:**
  JOIN queries, views, and aggregation functions are used to generate meaningful reports such as patient history and doctor workload.

Results and Screenshots

The Hospital Management System was successfully implemented and tested using PostgreSQL.

- All tables were created successfully with proper constraints
- Sample data was inserted without errors
- Stored procedure correctly added appointments
- Trigger automatically generated billing records after treatment insertion
- JOIN queries displayed accurate combined data
- Subqueries correctly identified high billing patients
- Views simplified complex data retrieval
- Transactions ensured safe and consistent data operations

The system produced reliable and accurate results, demonstrating the effectiveness of relational database design and SQL implementation.

Screenshots:

1. Most Prescribed Medicines

- **Uses GROUP BY + aggregation + percentage calculation**
- **Also uses window function (SUM() OVER())**

	medicine character varying (150)	times_prescribed bigint	pct_of_total numeric
1	Aspirin	1	14.29
2	Isosorbide Mononitrate	1	14.29
3	Sumatriptan	1	14.29
4	Amlodipine	1	14.29
5	Diclofenac	1	14.29
6	Ibuprofen	1	14.29
7	Metoprolol	1	14.29

2. Top Patients with Ranking

- **Uses COUNT + GROUP BY + DENSE_RANK()**
- **Ranking = very important concept**

	patient_name text	prescriptions_received bigint	rnk bigint
1	Anita Singh	2	1
2	Ravi Kumar	2	1
3	Meena Joshi	1	2
4	Suresh Gupta	1	2
5	Aakash Tiwari	1	2

3. Department Summary

- **Uses:**
  - **COUNT DISTINCT**
  - **FILTER**
  - **Percentage calculation**

	department character varying (100)	num_doctors bigint	total_appointments bigint	completed bigint	completion_rate_pct numeric
1	General Medicine	1	3	1	33.33
2	Cardiology	1	2	2	100.00
3	Neurology	1	2	1	50.00
4	Orthopedics	1	1	1	100.00
5	Pediatrics	1	1	0	0.00
6	Radiology	1	1	0	0.00

4. Patient Summary View

- **Uses:**
  - **JOIN**
  - **Aggregation**
  - **Derived column (age)**

	patient_id integer	patient_name text	age integer	blood_group character varying (5)	total_appointments bigint	total_records bigint
1	1	Ravi Kumar	41	B+	2	1
2	3	Suresh Gupta	47	A+	2	1

5. Doctor Availability + Function

- **Uses:**
  - **JOIN**
  - **Function call (fn_next_available_slot)**
  - **Real-world logic**

	doctor_id integer	doctor_name text	specialization character varying (100)	department character varying (100)	start_time time without time zone	end_time time without time zone	next_free_date date
1	1	Arjun Sharma	Cardiologist	Cardiology	09:00:00	17:00:00	2026-04-21
2	4	Sunita Nair	General Physician	General Medicine	09:00:00	17:00:00	2026-04-21
3	2	Priya Mehta	Neurologist	Neurology	09:00:00	17:00:00	2026-04-21
4	3	Rohit Verma	Orthopedic Surgeon	Orthopedics	09:00:00	17:00:00	2026-04-21
5	5	Deepak Patel	Pediatrician	Pediatrics	09:00:00	17:00:00	2026-04-21
6	6	Kavya Reddy	Radiologist	Radiology	09:00:00	17:00:00	2026-04-21

Future Scope

The Hospital Management System can be further improved with the following enhancements:

- **Performance Optimization:**
  Indexing can be applied to frequently used columns to improve query performance.

- **Improved Security:**
  Role-based access control can be implemented to restrict data access.

- **Backup and Recovery:**
  Database backup mechanisms can be added to prevent data loss.

- **Advanced Reporting:**
  More analytical queries and reports can be developed for better insights.

- **Scalability:**
  The system can be extended to support multiple hospitals or larger datasets.

- **Integration with Applications:**
  The database can be connected to web or mobile applications for real-world usage.

Conclusion

The Hospital Management System successfully demonstrates the practical implementation of advanced **Database**

Management System (DBMS) concepts using PostgreSQL. The project focuses on designing a well-structured relational database that efficiently manages hospital data such as patients, doctors, appointments, medical records, and prescriptions.

The database schema is designed using proper normalization techniques to reduce redundancy and ensure data consistency. Relationships between tables are maintained using primary and foreign keys, which enforce referential integrity. Advanced SQL features such as **views, functions, stored procedures, triggers, and analytical queries** have been effectively implemented.

The system also incorporates automation through triggers, validation through stored procedures, and analytical insights using complex queries such as aggregations, window functions, and CTEs. These features highlight the capability of PostgreSQL in handling real-world data processing and decision-making tasks.

Overall, the project provides a strong understanding of database design, query optimization, and data integrity management. It demonstrates how a well-designed database system can serve as the backbone of a real-world application like hospital management, ensuring efficiency, reliability, and scalability.

References

The following resources were used during the development of the Flight Route Planner System:

Books:

1. **Date, C. J.**
   An Introduction to Database Systems, Pearson.

2. **Silberschatz, A., Korth, H. F., & Sudarshan, S.**
   Database System Concepts, McGraw-Hill.

3. **Elmasri, R., & Navathe, S. B.**
   Fundamentals of Database Systems, Pearson Education.

Online Resources:

1. PostgreSQL Official Documentation
   https://www.postgresql.org/docs/

2. SQL Tutorial – GeeksforGeeks
   https://www.geeksforgeeks.org/sql-tutorial/

3. PostgreSQL Tutorial
   https://www.postgresqltutorial.com/